

Static and Shared libraries Lab

Objectives for Today's Lab

When developing an application, a larger application in particular, often external, open source libraries are used. In general, the phrase *external dependencies* is used to describe these various libraries. There are in general 3 options to use these libraries within your application:

1. Build static libraries (.a) and link them together with your object files (.o) into one executable
2. Build shared libraries (.so) and install them on a target system to be used by your application
3. Dynamically load a shared library (.so) at runtime

A combination of the options above is also possible.

1 Static linking

The advantage of static linking (option 1) is that is in general safe, all necessary dependencies are in one executable preventing run-time failures if not all dependencies are available on the target system. The GNU linker discards unused symbols after linking reducing code size, but this is not true for all linkers. The disadvantage is of course, if on the target system a whole suite of applications are running using the same external libraries, multiple copies of the same library are linked together into one executable, using extra disk space and memory if the applications are running simultaneously.

Building Library - static

- Navigate to lab directory

Run program without libraries

```
$ gcc -c main.c
$ gcc -c calc.c
$ gcc -o main calc.o main.o -lm or gcc -o main calc.o main.o /usr/lib/libm.a

$ ./main
sqrt(100)=10
```

Now, let's build a new library using **ar**.

Build Static library

```
$ ar crv libcalc.a calc.o
```

Now we can make executable from the library we just built, **libcalc.a**

Run with static Library linked

```
$ gcc -o main main.o libcalc.a -lm
or
$ gcc -o main main.o -L. -lcalc -lm

$ ./main
sqrt(100)=10
```

The **-L.** option tells compiler that the library, **libcalc.a** can be found in the current directory.

We can list symbols in the object/library by using **nm** command:

List executable symbols

```
$ nm main

080495a0 d __DYNAMIC
08049674 d __GLOBAL_OFFSET_TABLE__
0804855c R __IO_stdin_used
           w __Jv_RegisterClasses
08049590 d __CTOR_END__
0804958c d __CTOR_LIST__
08049598 D __DTOR_END__
08049594 d __DTOR_LIST__
08048588 r __FRAME_END__
0804959c d __JCR_END__
0804959c d __JCR_LIST__
08049694 A __bss_start
08049690 D __data_start
08048510 t __do_global_ctors_aux
08048380 t __do_global_dtors_aux
08048560 R __dso_handle
0804958c d __fini_array_end
0804958c d __fini_array_start
           w __gmon_start__
08048509 T __i686.get_pc_thunk.bx
0804958c d __init_array_end
0804958c d __init_array_start
08048490 T __libc_csu_fini
080484a0 T __libc_csu_init
           U __libc_start_main@@@GLIBC_2.0
0804958c d __preinit_array_end
0804958c d __preinit_array_start
08049694 A __edata
0804969c A __end
08048538 T __fini
08048558 R __fp_hw
080482c0 T __init
08048330 T __start
08048450 T calcSqrt
08048354 t call_gmon_start
08049698 b completed.5791
08049690 W data_start
08049694 b dtor_idx.5793
080483e0 t frame_dummy
08048404 T main
           U printf@@@GLIBC_2.0
           U sqrt@@@GLIBC_2.0
```

List library symbols

```
$ nm libcalc.a
calc.o:
00000000 T calcSqrt
           U sqrt
```

The character in the second column represents the symbol type: **T** refers to a text section symbol that is defined in the library, **U** refers to a symbol that is referenced by the library but is not defined by it. An uppercase letter specifies an external symbol, while a lowercase represents an internal symbol.

The string in the 3rd column provides the mangled symbol name. We can unmangle a symbol using **c++filt** command.

We can list global symbols using **g** option in the **nm** command:

```
nm -g main
```

Extra: show size and memory dump

2 Dynamic linking

With dynamic linking (option 2), this disadvantage is prevented by installing shared libraries (.so) onto the target system. This is often done by installing a package or by using an install target after building (for example, created by Automake). The .so files are installed in common directory locations monitored by the *ldconfig* cache such that all shared libraries can be loading at startup of your application. If shared objects are stored in other locations, the LD_LIBRARY_PATH environment variable can be set with additional directory locations of shared object files separated by a colon (:) so they can be resolved at startup of your application. Besides the fact the correct libraries must be present on the target system, your application must link against them **at linktime**. The location of a shared object may be different during linking than the target location on the target system. Especially after cross-compiling, you must make sure the application is linked with the correct, foreign version of a shared library.

Building Library - shared (dynamic)

- Navigate to lab directory

A typical shared library extension is **so**. For math library, it is **/usr/lib/libm.so**.

When a program uses shared library, the code itself is not included in the program, but it just links the library to refer it at run time. In that way, several program use the library without having it, and can save space. In other words, shared libraries address the disadvantages of static libraries. A shared library is an object module that can be loaded at run time at an arbitrary memory address, and it can be linked to by a program in memory. Shared libraries often are called as shared objects. On most UNIX systems they are denoted with a **.so** suffix and Microsoft refer to them as DLLs (dynamic link libraries).

Also, shared library has addition advantage over static library because it can be updated independently.

Actually, the file, **/usr/lib/libm.so** is a symbolic link to the revision of **libm.so.6**, and it is used at compile time.

```
$ ls -la libm.so
lrwxrwxrwx 1 root root 19 Feb  6 2008 libm.so -> ../../lib/libm.so.6
```

When linux application starts, the OS checks the library version requested by the application.

In linux system, **ld.so** (dynamic linker/loader) loads the shared libraries needed by a program, prepares the program to run, and then runs it.

When we try to run a program, but it is complaining that a library is missing though we know the library is there (such as in **/usr/local/lib**). But most likely **/usr/local/lib** is not on the default place (**/usr/lib**) where the program looks for the lib. So, we need to let the system know where to look for. There is a file (**/etc/ld.so.conf**) on our system where all the paths to the libraries are mentioned. To do that, we should run **ldconfig**.

We can check the required shared libraries for a program to run by using **ldd**:

```
$ ldd main
linux-gate.so.1 => (0x00dbb000)
libm.so.6 => /lib/libm.so.6 (0x493c9000)
libc.so.6 => /lib/libc.so.6 (0x4a439000)
/lib/ld-linux.so.2 (0x4a417000)
```

This our case, the **main** program is using shared libraries for math and Standard C.

To make shared library with **calc.o**, we do the following:

Build shared library

```
$ gcc -shared -fPIC -o libcalc.so calc.o
```

The command tells the compiler driver to generate a shared library, **libcalc.so**, from the object module **calc.o**.

The **-fPIC** option tells the compiler to generate **position independent code (PIC)**.

Now, suppose the primary object module is **main.o**, which has dependencies on **calc.o**. In this case, the linker is invoked with:

Link with shared library

```
$ gcc main.o ./libcalc.so -lm
```

The command above makes an executable file, **a.out**, in a form that can be linked to **libcalc.so** at load time.

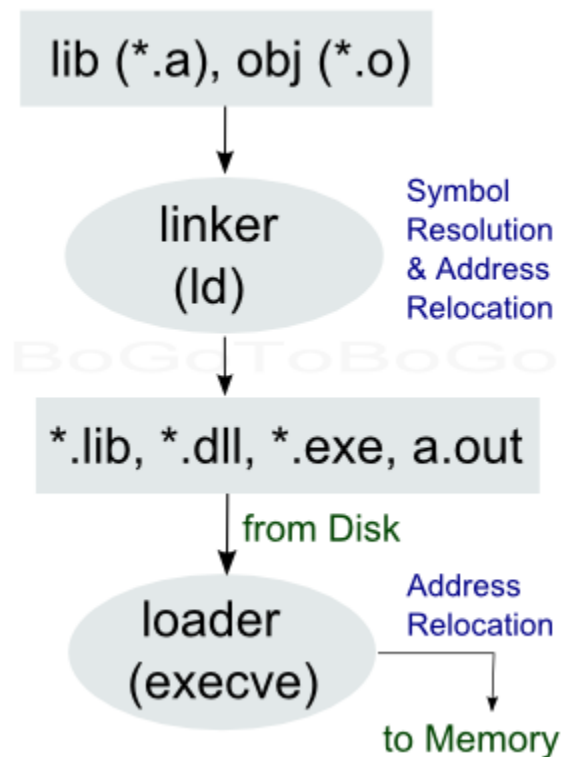
However, **a.out** does not contain the object module **calc.o**, which would have been included had we created a static library instead of a shared library. The executable simply contains some relocation and symbol table information that allow references to code and data in **libcalc.so** to be resolved at run time.

Thus, **a.out** here is a partially executable file that still has its dependency in **libcalc.so**.

The executable also contains a **.interp** section that has the name of the dynamic linker, which itself is a shared object on Linux systems, **ld-linux.so**. So, when the executable is loaded into memory, the loader passes control to the dynamic linker. The dynamic linker contains some start-up code that maps the shared libraries to the program's address space.

It then does the following:

1. relocates the text and data of **libcalc.so** into memory segment
2. relocates any references in **a.out** to symbols defined by **libcalc.so**



Extra: show size and memory dump

Then, the dynamic linker passes control to the application, and now the location of shared object is fixed in the memory.

Shared libraries can be loaded from applications even in the middle of their executions. An application can request a dynamic linker to load and link shared libraries, even without linking those shared libraries to the executable. Linux provides system calls, such as **dlopen** to load a shared object, **dlsym** to look up a symbol in that shared object, and **dlclose** to close the shared object.

We've already used some of the tools to explore object/executable files, here is the list of tools:

1. **ar**
creates static libraries.
2. **ldd**
lists the shared libraries on which the object binary is dependent.
3. **nm**
lists the symbols defined in the symbol table of an object file or a static library.
4. **objdump**
to display all the information in an object binary file.
5. **strings**
list all the printable strings in a binary file.
6. **strip**
deletes the symbol table information.

7. `c++filt`

demangle low-level names into user-level names (unix/linux command).

Linker vs Loader

Unlike some authors who prefer using combined word "link-loading, in the page, I've been using those separated from each other.

1. **Linker**

After compilation, the fragments (compilation unit) of a program are glued together by a linker.

Combines two or more objects (**relocatable object**) and supplies the information needed to allow references between them.

Relocation (loader does this as well) and the **resolution of external references** are performed by the linker.

2. **Loader**

Loading an **executable object** into **memory** for execution.

Note:

1. **relocatable object**

A relocatable object code is acceptable as input to a linker.

A relocatable object file has following table information: import table, relocation table, and export table.

2. **executable object**

Executable object code is acceptable as an input to a loader, and it can be brought into memory and run.

Executable object file is distinguished by the fact that it contains no references to external symbols, and it also defines a starting address for execution.

3 Dynamic loading

The 3rd option is dynamic loading shared objects during the runtime scope of an application. This is done by invoking calls to [dlopen\(\)](#), [dlclose\(\)](#), [dlsym\(\)](#) and [dLError\(\)](#) in your application. In general, I would **strongly discourage dynamically loading** of shared objects **without a specific use-case** to do so.

Dynamic loading requires some knowledge and skills and pitfalls are not uncommon:

- Dynamic loading requires C linkage with ugly pointer to function pointer casts reducing readability and re-usability
- It relies on dynamic polymorphism which must be used correctly to avoid linking errors
- When your application tries to load a shared object which cannot be found, your application will most likely crash. This may happen after startup of your program.
- A shared object must have been loaded during the scope of the operations you are performing. If a shared object is unloaded when you call polymorphic methods on objects in this shared library, your application will just crash with a segmentation fault for which the GNU debugger will not be very helpful.

To illustrate how you would use dynamic loading with C++, you can start making a simple dynamic library:

Simple dynamic library

```
#include <memory>

class ISharedLibraryImp
{
public:
    virtual ~ISharedLibraryImp() {}

    virtual void printDictionaryContents() noexcept = 0;
};

class SharedLibraryImp : public ISharedLibraryImp
{
public:
    SharedLibraryImp() = default;
    ~SharedLibraryImp();

    void printDictionaryContents() noexcept;
};

extern "C"
{
    void test();

    std::shared_ptr<ISharedLibraryImp> create();
}
```

```
}
```

First, note the two C functions **test()** and **create()** which are declared with extern C linkage. This is needed for backwards compatibility with C. These functions are dynamically located with **dlsym()** inside a shared object (.so) when it is loaded. With normal C++ linkage, **dlsym()** will not be able to resolve the function.

Secondly, note that the implementation is derived from an interface **ISharedLibraryImp** with the pure virtual method **printDictionaryContents()**. The content of this method is not important at this point, but if it was not a pure virtual method, the GNU linker wants to resolve this symbol during link-time which will, of course, fail with a link error 'unresolved symbols'.

The interface of a class dynamically loading an interface could look like this: (using gcc version 5.4.0 with C++14)

Dynamic Loader class

```
class DynamicLoader
{
public:
    static constexpr int ERROR_BUF_SIZE = 80;

    DynamicLoader(const char * filename, int flags = RTLD_NOW);
    DynamicLoader(const DynamicLoader& src) = delete;
    virtual ~DynamicLoader();

    void open();
    void close();

    void* lookup(const char* symbolStr);

    void throwException(const char* errorStr);

private:
    const std::string m_filename;
    const int         m_flags;
    void*             m_dynamicHandle;

};
```

Upon construction, the loader knows which filename to load. The flag **RTLD_NOW** can be modified, but I recommend only using this default flag to ensure all symbols are resolved after loading the shared object to avoid surprises later.

The method **open()** will attempt to load the shared object and **close()** will unload the shared object from memory. The lookup method will attempt to lookup a function inside this shared object. It is recommended to throw a specific exception in case the library could not be found or a symbol cannot be loaded to indicate something went wrong with dynamic loading:

Dynamic Loader class impl

```
DynamicLoader::DynamicLoader(const char * filename, int flags)
: m_filename(filename),
  m_flags(flags),
  m_dynamicHandle(nullptr)
{
}

DynamicLoader::~DynamicLoader()
{
    close();
}

void DynamicLoader::open()
{
    m_dynamicHandle = dlopen(m_filename.c_str(), m_flags);

    if (0 == m_dynamicHandle)
    {
        throwException(dlerror());
    }
}
```

```

    dlerror(); // Clear any existing error
}

void DynamicLoader::close()
{
    if (nullptr != m_dynamicHandle)
    {
        dlclose(m_dynamicHandle);
    }
}

void* DynamicLoader::lookup(const char* symbolStr)
{
    void *testFunc = nullptr;
    testFunc = dlsym(m_dynamicHandle, symbolStr);

    if (0 == testFunc)
    {
        throwException(dlerror());
    }

    return testFunc;
}

void DynamicLoader::throwException(const char* errorStr)
{
    char errorBuf[ERROR_BUF_SIZE];
    memset(errorBuf, '\0', sizeof(errorBuf));

    snprintf(errorBuf, sizeof(errorBuf), "%s\n", errorStr);
    throw DynamicLoadingException(errorStr);
}

```

Note that when an object of class `DynamicLoader` goes out of scope, the corresponding shared object file is also unloaded (**`dlclose()`** call). Make sure you perform only relevant calls to the shared object files during the scope of the `DynamicLoader` to avoid crashes.

The **`lookup()`** method tries to resolve a symbol with a particular signature. The result is a void pointer which must be casted to a pointer to a function with the correct signature using **`reinterpret_cast<>()`**. A **main** function in your application loading and invoking the **`test()`** and **`create()`** functions could look like this:

Test App

```

#include <stdio.h>

#include <iostream>
#include <cstdlib>
#include <memory>

#include "DynamicLoader.h"
#include "DynamicLoadingException.h"
#include "SharedLibraryImp.h"

int main(int argc, char* argv[])
{
    DynamicLoader loader("/home/ahmealy/libDynamicLibraryExample.so", RTLD_NOW);
    void (*testFunc)();
    std::shared_ptr<SharedLibraryImp> (*create)();

    try
    {
        loader.open();

        testFunc = reinterpret_cast<void (*)(>> (loader.lookup("test"));
        testFunc();

        create = reinterpret_cast<std::shared_ptr<SharedLibraryImp> (*)(>> (loader.lookup("create"));
        std::shared_ptr<SharedLibraryImp> imp = create();
        imp->printDictionaryContents();
    }
}

```

```

    }
    catch(const DynamicLoadingException& e)
    {
        std::cerr << e.what() << std::endl;
    }

    return EXIT_SUCCESS;
}

```

After casting the result of the **lookup()** function to the correct function pointer signatures, the functions can be invoked. Note that the header file **SharedLibraryImp.h** of the shared object is included to resolve used symbols at compile time. The method invocation

```
imp->printDictionaryContents();
```

is a call to a pure virtual function. Non-virtual methods of the class **SharedLibraryImp** cannot be called. So it is highly recommended to use the **interface segmentation** principle.

Also note the **imp** object, which is a shared pointer, goes out of scope before the loader instance. The reference count of the shared pointer will reach 0 first, cleanup up resources of the **imp** object. Then, the loader goes out of scope, resulting of unloading the shared object. If this was accidentally done the other way around, your program will most likely crash on exit. In large programs, crashes due to these scope problems may not be trivial.

When is dynamic loading useful?

In the beginning, I stated that I would discourage use of dynamic loading if it has no clear benefit. In general, if you want to be able to replace an implementation after you have installed the program. For example, if you have a version of a license mechanism implementation which you want to replace with a new version without having to release and reinstall the application on a target. Or you want to be able to modify or extend some functionality for particular users without having to distribute different packages where errors can be introduced when the incorrect package is installed accidentally.

4 Platform specific issues in loading dynamic libraries

Description

Loading a shared library differ between platforms. This leads to compilation errors when trying to compile same code for windows.

Linux

Linux platform uses from the `<dlfcn.h>` header file the functions (`dlopen`, `dlsym`, `dlclose`, `dlerror`) to load a Shared Object library (.so file).

Windows

Windows platform uses from the `<windows.h>` header file the functions (`LoadLibrary`, `GetProcAddress`, `FreeLibrary`, `GetLastError`) to load a Dynamic Loaded Library (.dll file).

Suggested Fix

We could use one of two approaches:

- Create a platform independent wrapper utility that abstracts loading shared library from the underlying platform.
- Use Boost.DLL
 - high level API
 - low level API.

Details: [Portable Loading of Shared Libraries](#) document

Further Reading:

<https://www.bogotobogo.com/cplusplus/libraries.php>

<https://www.internalpointers.com/post/library-linking-c-cpp>

<https://amir.rachum.com/blog/2016/09/17/shared-libraries/>

<https://docencia.ac.upc.edu/FIB/USO/Bibliografia/unix-c-libraries.html>

<https://www.linkedin.com/pulse/static-dynamic-libraries-c-nicolas-diego-bel%C3%A9n-de-lilla>

<https://web.cs.ucla.edu/classes/fall18/cs35L/assign/assign7.html>

<https://tldp.org/HOWTO/Program-Library-HOWTO/index.html>

<https://ocw.cs.pub.ro/courses/cns/labs/lab-02>